

Lecture 05: Solving 8 Problems

Is lecture mein hum **8 fundamental programming problems** ko solve karenge — Prime Number, Factorial, Sum of Digits, Reverse a Number, Decimal to Binary, Binary to Decimal, Fibonacci Number, aur Square Root. Saath mein **do-while loop** ka concept bhi cover karenge. Ye problems interviews mein sabse zyada puche jaate hain.



Do-While Loop

INTUITION

do-while loop mein condition **baad mein check hoti hai** — matlab loop body kam se kam ek baar toh chalegi hi. while loop mein condition pehle check hoti hai, toh agar condition false ho toh loop ek baar bhi nahi chalega.

```
int i = 1; // Initialize

do {
    cout << i << " "; // Print
    i++;               // Increment / Decrement
} while(i < 10);       // Condition check (after body)
```

WORKFLOW

- **Step 1:** Loop body execute karo
- **Step 2:** Condition check karo
- **Step 3:** Agar condition true hai, toh Step 1 repeat karo
- **Step 4:** Agar condition false hai, loop exit karo

IMPORTANT

do-while loop ka use tab karo jab aapko **kam se kam ek baar execution** guaranteed chahiye. Jaise user se input lena jab tak valid na ho, ya menu-driven programs.



Problem 1: Prime Number Check

INTUITION

Prime number wo number hota hai jo **sirf 1 aur apne aap se divisible** ho. Sabse pehle check karo ki number 2 se chhota hai ya nahi — kyunki 2 se chhoti values (0, 1) prime nahi hoti. Phir 2 se lekar num-1 tak har number se divide karke check karo.

APPROACH

BASIC

OPTIMIZED

```
int num;

cout << "Enter the Number: ";
cin >> num;

// 2 se chhoti value kabhi bhi prime nahi hoti
if (num < 2) {
    cout << num << " is Not a Prime Number";
    return 0;
}

// 2 se num-1 tak divide karke check karo
for (int i = 2; i < num; i++) {
    if (num % i == 0) {
        cout << num << " is Not a Prime Number";
        return 0;
    }
}

cout << num << " is a Prime Number" << endl;
```

PERFORMANCE

Is approach ki **time complexity $O(n)$** hai. Optimized approach mein sirf $\sqrt{\text{num}}$ tak check karte hain — **$O(\sqrt{n})$** complexity. Kyunki agar n ka koi factor $\sqrt{n}$ se bada hai, toh uska corresponding pair $\sqrt{n}$ se chhota hoga.



Problem 2: Factorial

INTUITION

Factorial ka matlab hai ek number se lekar 1 tak sabka product. Jaise $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. Base cases: $0! = 1$ aur $1! = 1$. Loop 1 se n tak chalao aur har iteration mein running product update karo.

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
```

```
int fact = 1;

for(int i = 1; i ≤ 5; i++) {
    fact = fact * i;
}

cout << fact << endl; // 120
```

EDGE CASE

Integer Overflow : int ki range takriban 2 billion hoti hai. $13! = 6,227,020,800$ jo ki int ki range se bahar hai. Bade factorials ke liye long long use karo, ya phir **modulo** lete raho agar sirf remainder chahiye.



Problem 3: Sum of Digits

INTUITION

Number ke har digit ko alag karna hai aur unka sum nikalna hai. **Modulo 10** se last digit nikalti hai, aur **divide by 10** se last digit remove hoti hai. Ye process tab tak repeat karo jab tak number 0 nahi ban jata.

WORKFLOW

- **Step 1:** number % 10 se last digit nikalo
- **Step 2:** Us digit ko sum mein add karo
- **Step 3:** number / 10 se last digit remove karo
- **Step 4:** Jab tak number > 0, repeat karo

```
int number = 12345;
int sum = 0;

while (number > 0) {
    int lastDigit = number % 10; // Last digit nikalo

    sum = sum + lastDigit;        // Sum me add karo

    number = number / 10;        // Last digit remove karo
}

cout << "Sum of Digits: " << sum << endl; // 15
```

Key Insight: **Modulo 10** hamesha last digit deta hai, aur **Divide by 10** hamesha last digit ko remove kar deta hai. Ye trick digit manipulation problems mein universally kaam aati hai.



Problem 4: Reverse a Number

INTUITION

Number ko reverse karne ka formula hai: **ans = ans * 10 + lastDigit**. Har baar ans ko 10 se multiply karke usme last digit add karo. Jaise 12345 ko reverse karne pe 54321 aata hai.

```
int num = 12345;
int ans = 0;

while (num > 0) {
    int lastDigit = num % 10;
    num = num / 10;
    ans = ans * 10 + lastDigit;
}

cout << ans << endl; // 54321
```

WORKFLOW

Iteration	lastDigit	num	ans
Initial	-	12345	0
1	5	1234	5
2	4	123	54
3	3	12	543
4	2	1	5432
5	1	0	54321



Problem 5: Decimal to Binary

INTUITION

Decimal number ko binary mein convert karne ke liye number ko **baar-baar 2 se divide** karo aur har baar **remainder** ko store karo. Remainders ko **reverse order** mein padhne pe binary number milta hai.

```
2 | 13 → 1 ↑
2 | 6  → 0 ↑
2 | 3  → 1 ↑
2 | 1  → 1 ↑
    0
```

Binary: 1101

```

int num = 13;
int ans = 0;
int mul = 1; // multiplier (10^0, 10^1, 10^2...)

while(num != 0) {
    int rem = num % 2; // remainder (0 ya 1)
    num = num / 2; // number ko 2 se divide karo
    ans = ans + rem * mul; // remainder ko correct position pe place karo
    mul = mul * 10; // next decimal place ke liye
}

cout << ans << endl; // 1101

```

WORKFLOW

- **Step 1:** Number ko 2 se divide karo, remainder nikalo
- **Step 2:** Remainder ko ans mein add karo with proper place value
- **Step 3:** mul ko 10 se multiply karo (next digit position)
- **Step 4:** Number ko 2 se divide karke update karo
- **Step 5:** Jab number 0 ho jaye, loop exit karo

EDGE CASE

XOR Approach: Kuch log is problem ko XOR operation se solve karte hain, lekin wo approach **code readable nahi hoti** aur interviews mein explain karna mushkil hota hai. Hamesha simple, readable approach prefer karo.



Problem 6: Binary to Decimal

INTUITION

Binary number ko decimal mein convert karne ke liye har digit ko uski **position ke power of 2** se multiply karo. Rightmost digit 2^0 , usse left 2^1 , phir 2^2 , aur aage badhte jao.

Binary: 1 1 0 1

↓ ↓ ↓ ↓

8 4 2 1 ← 2^3 2^2 2^1 2^0

$$1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 8 + 4 + 0 + 1 =$$

13

```

int num = 1101; // binary number stored as int
int ans = 0;
int mul = 1; // 2^0, 2^1, 2^2...

while(num != 0) {
    int rem = num % 10; // last digit nikalo (0 ya 1)
    num = num / 10; // last digit remove karo
    ans = ans + rem * mul; // digit * power of 2
    mul = mul * 2; // next power of 2
}

cout << ans << endl; // 13

```

Key Insight: Decimal to Binary aur Binary to Decimal dono mein **same core logic** hai — *num % 10 se digit nikalo, num / 10 se remove karo. Bas mul ka base alag hai: Decimal→Binary mein mul × 10 (place value), Binary→Decimal mein mul × 2 (power of 2).*



Problem 7: Fibonacci Number

INTUITION

Fibonacci series mein **har number apne pehle do numbers ka sum** hota hai. Base cases: $F(0) = 0$ aur $F(1) = 1$. Recursive formula: $F(n) = F(n-1) + F(n-2)$.

$F(0) = 0 \rightarrow$ Base Case
 $F(1) = 1 \rightarrow$ Base Case
 $F(2) = F(1) + F(0) = 1 + 0 = 1$
 $F(3) = F(2) + F(1) = 1 + 1 = 2$
 $F(4) = F(3) + F(2) = 2 + 1 = 3$
 $F(5) = F(4) + F(3) = 3 + 2 = 5$
 $F(6) = F(5) + F(4) = 5 + 3 = 8$

Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

```
int fib(int n) {
    if(n ≤ 1) {
        return n; // Base case: F(0)=0, F(1)=1
    }

    int first = 0, second = 1;

    for(int i = 2; i ≤ n; i++) {
        int curr = first + second;
        first = second;
        second = curr;
    }

    return second;
}
```

WORKFLOW

- **Step 1:** Base cases handle karo ($n \leq 1$)
- **Step 2:** $first = 0, second = 1$ initialize karo
- **Step 3:** Loop 2 se n tak chalao
- **Step 4:** $curr = first + second$ calculate karo
- **Step 5:** $first = second$ aur $second = curr$ update karo
- **Step 6:** Return second (jo ki $F(n)$ hai)

PERFORMANCE

Iterative approach ki **time complexity $O(n)$** aur **space complexity $O(1)$** hai. Recursive approach ki time complexity **$O(2^n)$** hoti hai (exponential), isliye iterative hamesha prefer karo. Recursive version mein **TLE (Time Limit Exceeded)** ho sakta hai competitive programming mein.



Problem 8: Square Root of a Number

INTUITION

Square root wo number hota hai jiska **square original number ke barabar** ho. Jaise $\sqrt{16} = 4$ kyunki $4 \times 4 = 16$. Approach: 1 se lekar number tak har value ka square check karo, jahan $i \times i > x$ ho jaye, wahan $i-1$ return karo.

$$\sqrt{16} = 4 \rightarrow 4 \times 4 = 16$$

$$\sqrt{25} = 5 \rightarrow 5 \times 5 = 25$$

$$\sqrt{6} = 2 \rightarrow 2 \times 2 = 4 \leq 6, 3 \times 3 = 9 > 6$$

Isliye integer part 2 hai

```
int mySqrt(int x) {  
    if(x == 0) return 0;  
  
    int ans = 1;  
  
    for(int i = 1; i <= x; i++) {  
        if((long long)i * i <= x) {  
            ans = i;  
        } else {  
            break;  
        }  
    }  
  
    return ans;  
}
```

EDGE CASE

Integer Overflow: $i * i$ mein agar i bada ho toh int overflow ho sakta hai. Isliye $(\text{long long})i * i$ use karo taaki multiplication long long mein ho. Ya phir $i \leq x / i$ condition use karo — ye overflow se bachata hai.

PERFORMANCE

Linear search approach ki **time complexity $O(n)$** hai. Optimized approach mein **Binary Search** use karte hain jo **$O(\log n)$** complexity deta hai. Interview mein pehle linear approach batao, phir optimized binary search version batao.



Bonus: Armstrong Number

INTUITION

Armstrong number (ya Narcissistic number) wo number hota hai jisme **har digit ka cube sum** usi number ke barabar hota hai. 3-digit numbers ke liye: sum of cubes of digits = original number.

153: $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ ✓ Armstrong

163: $1^3 + 6^3 + 3^3 = 1 + 216 + 27 = 244$ ✗ Not Armstrong

264: $2^3 + 6^3 + 4^3 = 8 + 216 + 64 = 288$ ✗ Not Armstrong

```
int num = 153;
int original = num;
int sum = 0;

while(num > 0) {
    int digit = num % 10;
    sum = sum + (digit * digit * digit); // cube of digit
    num = num / 10;
}

if(sum == original) {
    cout << "Armstrong Number" << endl;
} else {
    cout << "Not Armstrong" << endl;
}
```

BEST PRACTICE

Original number ko **ek alag variable mein store** karo (`original = num`) kyunki loop ke andar `num` ki value 0 ho jati hai. Comparison ke liye original value chahiye hoti hai.



Key Takeaways

TAKEAWAY

- **Do-While Loop:** Condition baad mein check hoti hai, body kam se kam ek baar chalti hai. Menu-driven programs mein useful.
- **Prime Check:** 2 se $n-1$ tak check karo. Optimized: $\sqrt{n}$ tak check karo. Edge case: numbers < 2 are not prime.
- **Factorial:** Running product approach. Overflow ka dhyan rakhna — use long long for large values.
- **Sum of Digits:** $\text{num} \% 10 \rightarrow$ last digit, $\text{num} / 10 \rightarrow$ remove last digit. Universal digit manipulation trick.
- **Reverse Number:** $\text{ans} = \text{ans} * 10 + \text{lastDigit}$. Har iteration mein place value shift hoti hai.
- **Decimal $\leftrightarrow$ Binary:** Same core logic, bas multiplier alag — $\times 10$ for decimal place value, $\times 2$ for power of 2.
- **Fibonacci:** Iterative approach $O(n)$ time, $O(1)$ space. Recursive $O(2^n)$ — avoid in interviews.
- **Square Root:** Linear search $O(n)$, Binary Search $O(\log n)$. Overflow se bachne ke liye long long ya $i \leq x / i$ use karo.
- **Armstrong:** Sum of cubes of digits = original number. Original value ko alag store karo.

Problem	Time Complexity	Space Complexity	Key Trick
Prime Check	$O(n) / O(\sqrt{n})$	$O(1)$	2 se $\sqrt{n}$ tak check
Factorial	$O(n)$	$O(1)$	Running product
Sum of Digits	$O(\text{digits})$	$O(1)$	$\%10 / /10$
Reverse Number	$O(\text{digits})$	$O(1)$	$\text{ans} * 10 + \text{digit}$
Decimal $\rightarrow$ Binary	$O(\log n)$	$O(1)$	Divide by 2, store remainder
Binary $\rightarrow$ Decimal	$O(\text{digits})$	$O(1)$	digit $\times$ power of 2
Fibonacci	$O(n)$	$O(1)$	Two variables swap
Square Root	$O(n) / O(\log n)$	$O(1)$	$i * i \leq x$, avoid overflow

INTERVIEW TIP

Ye 8 problems **interviewers ki favorite list** mein hain. Hamesha pehle brute force approach batao, phir optimized version batana. Jaise Prime Check mein pehle $O(n)$ batao, phir $O(\sqrt{n})$. Square Root mein pehle linear search, phir binary search. Ye dikhata hai ki aap optimization soch sakte ho.

Key Insight: *Logic → Code → Initial Phase → 1 Month* . Pehle logic samajhna zaroori hai, phir code likhna, phir practice. In 8 problems ko ek baar samajh lo, toh aage ke complex problems bhi asaani se solve kar paoge.

MADE BY HARSHAL CHAUHAN